



RUHR-UNIVERSITÄT BOCHUM

Fundamentals of GPU Programming

Lecture 8

Denis Eremin

Fakultät für Elektrotechnik und Informationstechnik
Lehrstuhl für Theoretische Elektrotechnik

Optimization Strategies for the Reduction

We will learn

- Improvement of an algorithm by changing the control flow (removing thread divergence issues)

General Remarks on Reduction

- Common and important data primitive
- Easy to implement in CUDA, hard to get it right
- Many optimization opportunities, good example of fine-tuning techniques

Source

M. Harris, *Optimizing Parallel Reduction with CUDA*

Programming Massively Parallel Processors: A Hands-On Approach
(Hwu, Kirk, Hajj) 4rd Ed.

Reduction

Reduction uses binary operator on a data set to yield a single result, e.g.,

$$S = \sum_n a_n \quad \text{or} \quad a_{max} = \max_n \{a_n\}$$

max

min

addition

multiplication

or a user-defined operator, which is
associative & commutative and
has a well-defined unity value

Reduction, Sequential Implementation

```
void reduction( float output, float *input, int length)
{
    output = 0;
    for (int j=1; j<length; ++j)
    {
        output += input[j-1]; // min(output, input[j-1]);
    }
}
```

Work complexity $O(n)$

- we want to have same work complexity for the parallel algorithm (work efficient algorithm)

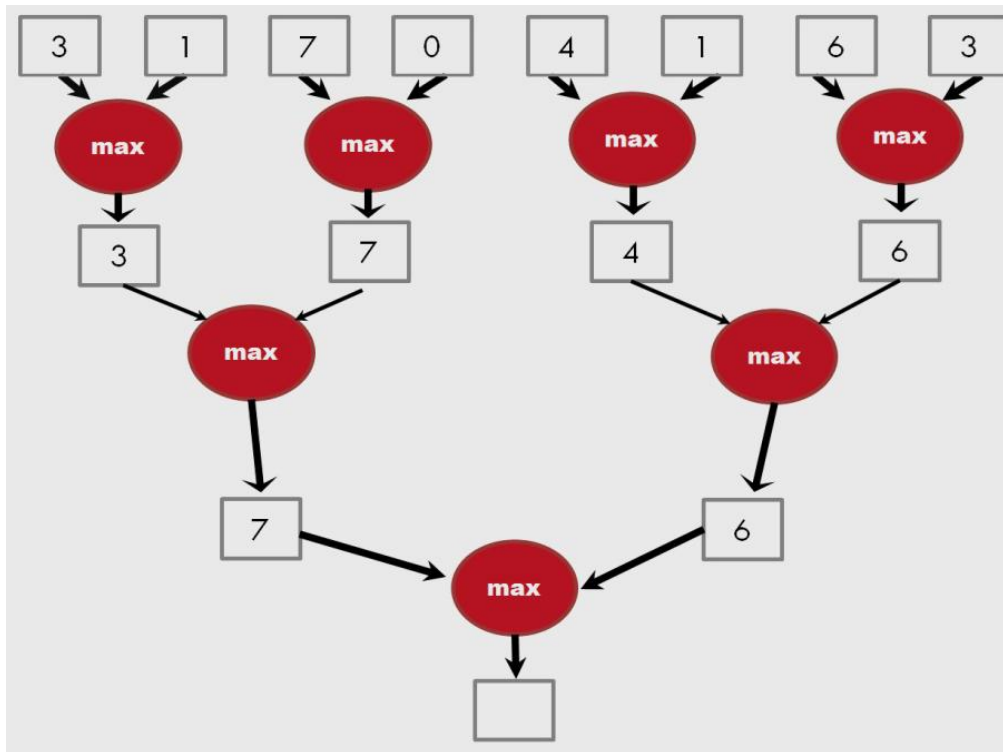
Parallelization of Reduction

If the operator is commutative and associative,
one can implement a parallel version of the reduction algorithm

Parallelization strategy:

- Partition the input data set into chunks, each to be processed by a thread block
- Use a balanced binary tree reduction algorithm to perform the reduction inside each of the thread blocks

Toy (Small Scale) Example



8 input elements, 3
steps (tree levels),
7 operations

Efficiency Analysis

For n leaves there are $\log_2(n)$ levels,

each level $d = [0, \log_2(n))$ has 2^d nodes

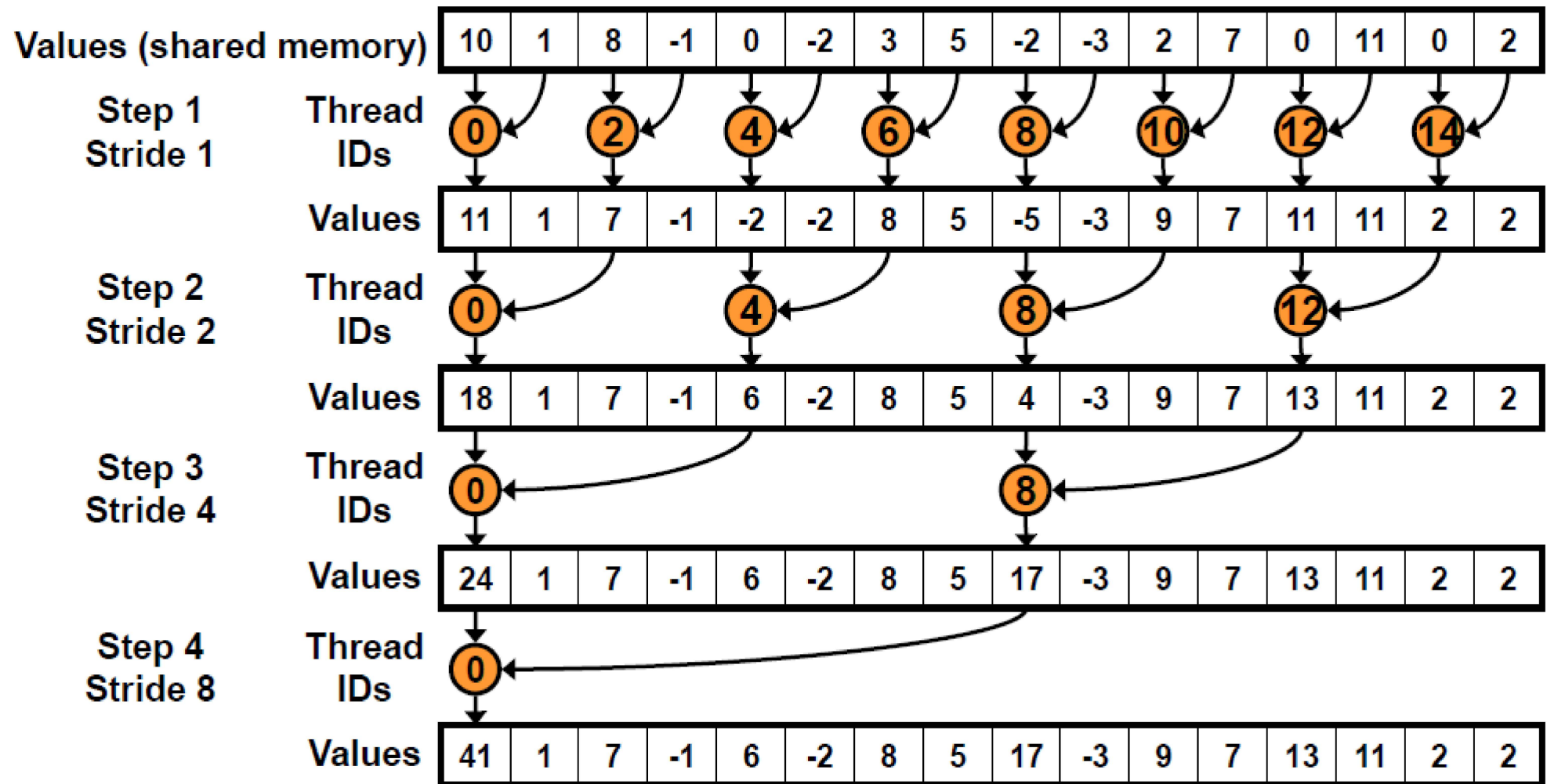
Work complexity $\sum_0^{\log_2(n)} 2^d = n - 1 = \mathcal{O}(n)$

Average parallelism $(n - 1) / \log(n - 1)$

For $n = 10^6$ average parallelism 50000

Reduction #1: Interleaved Addressing

```
__global__ void reduce0(int *g_idata, int *g_odata) {  
    extern __shared__ int sdata[];  
  
    // each thread loads one element from global to shared mem  
    unsigned int tid = threadIdx.x;  
    unsigned int i = blockIdx.x*blockDim.x + threadIdx.x;  
    sdata[tid] = g_idata[i];  
    __syncthreads();  
  
    // do reduction in shared mem  
    for(unsigned int s=1; s < blockDim.x; s *= 2) {  
        if (tid % (2*s) == 0) {  
            sdata[tid] += sdata[tid + s];  
        }  
        __syncthreads();  
    }  
  
    // write result for this block to global mem  
    if (tid == 0) g_odata[blockIdx.x] = sdata[0];  
}
```



Reduction #1: Interleaved Addressing

```
__global__ void reduce0(int *g_idata, int *g_odata) {  
    extern __shared__ int sdata[];  
  
    // each thread loads one element from global to shared mem  
    unsigned int tid = threadIdx.x;  
    unsigned int i = blockIdx.x*blockDim.x + threadIdx.x;  
    sdata[tid] = g_idata[i];  
    __syncthreads();  
  
    // do reduction in shared mem  
    for(unsigned int s=1; s < blockDim.x; s *= 2) {  
        if (tid % (2*s) == 0) {  
            sdata[tid] += sdata[tid + s];  
        }  
        __syncthreads();  
    }  
  
    // write result for this block to global mem  
    if (tid == 0) g_odata[blockIdx.x] = sdata[0];  
}
```

Divergent warps and slow
% operator are inefficient!

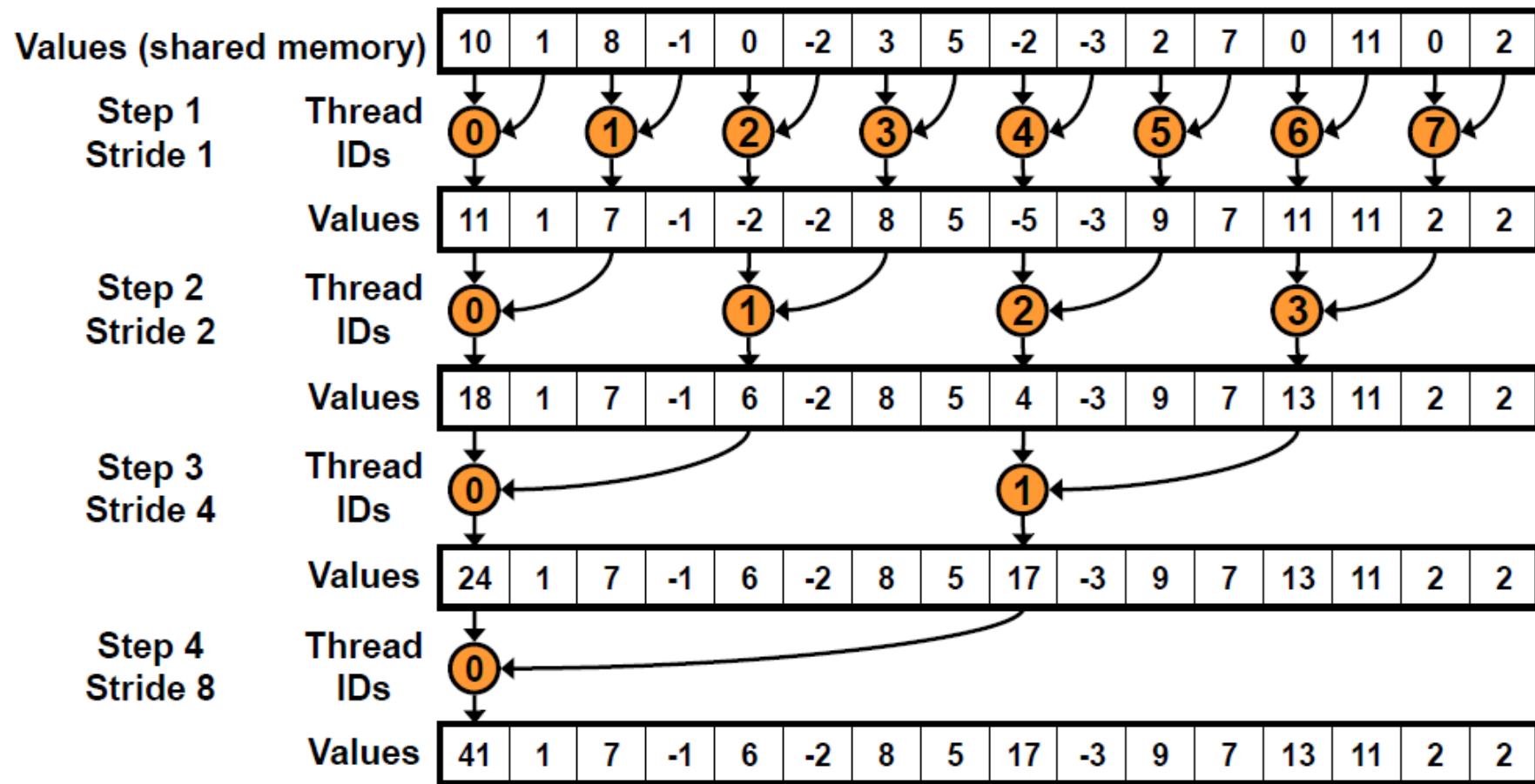
Reduction #2: Interleaved Addressing

Just replace divergent branch in inner loop:

```
for (unsigned int s=1; s < blockDim.x; s *= 2) {  
    if (tid % (2*s) == 0) {  
        sdata[tid] += sdata[tid + s];  
    }  
    __syncthreads();  
}
```

With strided index and non-divergent branch:

```
for (unsigned int s=1; s < blockDim.x; s *= 2) {  
    int index = 2 * s * tid;  
  
    if (index < blockDim.x) {  
        sdata[index] += sdata[index + s];  
    }  
    __syncthreads();  
}
```



New problem: shared memory conflicts!

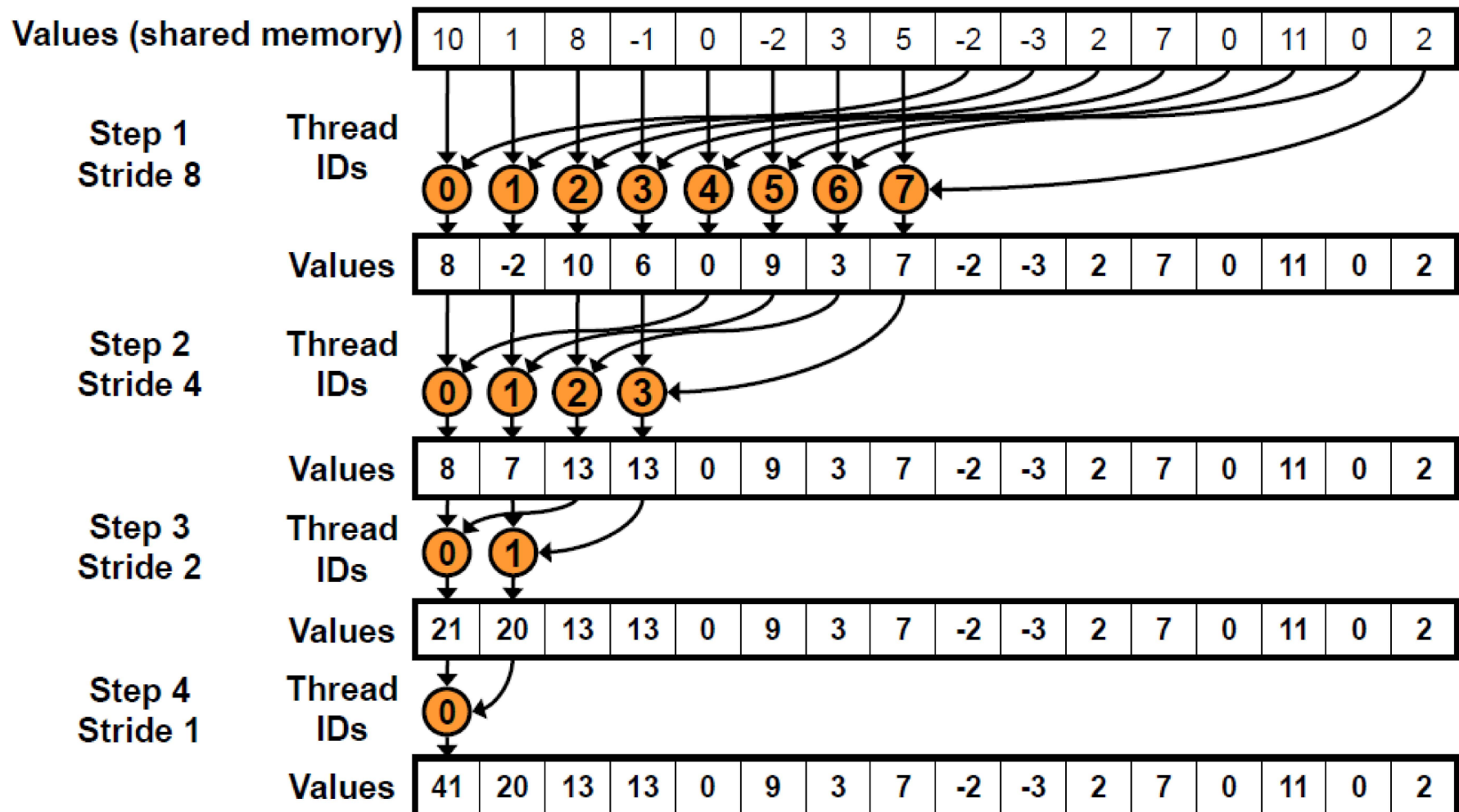
Reduction #3: Sequential Addressing

Just replace strided indexing in inner loop:

```
for (unsigned int s=1; s < blockDim.x; s *= 2) {  
    int index = 2 * s * tid;  
  
    if (index < blockDim.x) {  
        sdata[index] += sdata[index + s];  
    }  
    __syncthreads();  
}
```

With reversed loop and threadID-based indexing:

```
for (unsigned int s=blockDim.x/2; s>0; s>>=1) {  
    if (tid < s) {  
        sdata[tid] += sdata[tid + s];  
    }  
    __syncthreads();  
}
```




```
for (unsigned int s=blockDim.x/2; s>0; s>>=1) {  
    if (tid < s) {  
        sdata[tid] += sdata[tid + s];  
    }  
    __syncthreads();  
}
```

Problem: half of the threads are idle on the first iteration!

Reduction #4: First Add During Load

Halve the number of blocks, and replace single load:

```
// each thread loads one element from global to shared mem
unsigned int tid = threadIdx.x;
unsigned int i = blockIdx.x*blockDim.x + threadIdx.x;
sdata[tid] = g_idata[i];
__syncthreads();
```

With two loads and first add of the reduction:

```
// perform first level of reduction,
// reading from global memory, writing to shared memory
unsigned int tid = threadIdx.x;
unsigned int i = blockIdx.x*(blockDim.x*2) + threadIdx.x;
sdata[tid] = g_idata[i] + g_idata[i+blockDim.x];
__syncthreads();
```

Tackling Instruction Overhead Issues

1.

When active threads fall inside a single warp,
one can unroll instructions in that warp:

instructions are SIMD synchronous inside a warp
(one no longer needs `__syncthreads()` and `if (s<tid)`)

Reduction #5: Unroll the Last Warp

```
__device__ void warpReduce(volatile int* sdata, int tid) {  
    sdata[tid] += sdata[tid + 32];  
    sdata[tid] += sdata[tid + 16];  
    sdata[tid] += sdata[tid + 8];  
    sdata[tid] += sdata[tid + 4];  
    sdata[tid] += sdata[tid + 2];  
    sdata[tid] += sdata[tid + 1];  
}
```

↑

IMPORTANT:
For this to be correct,
we must use the
“volatile” keyword!

```
// later...  
for (unsigned int s=blockDim.x/2; s>32; s>>=1) {  
    if (tid < s)  
        sdata[tid] += sdata[tid + s];  
    __syncthreads();  
}  
  
if (tid < 32) warpReduce(sdata, tid);
```

Note: This saves useless work in *all* warps, not just the last one!
Without unrolling, all warps execute every iteration of the for loop and if statement

2.

Actually, we can completely unroll the for loop using the fact that number of threads in a thread block is limited and can only be a power of two number in the reduction algorithm.

To be generic, one can use templates for handling the blocks with unknown size at the compile time and along with the **switch** statement to choose during the run-time execution from different options, pre-defined at the compile time

Specify block size as a template parameter

```
template <unsigned int blockSize>  
__global__ void reduce5(int *g_idata, int *g_odata)
```

Reduction #6: Completely Unrolled

```
Template <unsigned int blockSize>
__device__ void warpReduce(volatile int* sdata, int tid) {
    if (blockSize >= 64) sdata[tid] += sdata[tid + 32];
    if (blockSize >= 32) sdata[tid] += sdata[tid + 16];
    if (blockSize >= 16) sdata[tid] += sdata[tid + 8];
    if (blockSize >= 8) sdata[tid] += sdata[tid + 4];
    if (blockSize >= 4) sdata[tid] += sdata[tid + 2];
    if (blockSize >= 2) sdata[tid] += sdata[tid + 1];
}
```

```
if (blockSize >= 512) {
    if (tid < 256) { sdata[tid] += sdata[tid + 256]; } __syncthreads(); }
if (blockSize >= 256) {
    if (tid < 128) { sdata[tid] += sdata[tid + 128]; } __syncthreads(); }
if (blockSize >= 128) {
    if (tid < 64) { sdata[tid] += sdata[tid + 64]; } __syncthreads(); }

if (tid < 32) warpReduce<blockSize>(sdata, tid);
```

Statements in red color are evaluated at the compile time

The "switch" statement

```
switch (threads)
{
  case 512:
    reduce5<512><<< dimGrid, dimBlock, smemSize >>>(d_idata, d_odata); break;
  case 256:
    reduce5<256><<< dimGrid, dimBlock, smemSize >>>(d_idata, d_odata); break;
  case 128:
    reduce5<128><<< dimGrid, dimBlock, smemSize >>>(d_idata, d_odata); break;
  case 64:
    reduce5< 64><<< dimGrid, dimBlock, smemSize >>>(d_idata, d_odata); break;
  case 32:
    reduce5< 32><<< dimGrid, dimBlock, smemSize >>>(d_idata, d_odata); break;
  case 16:
    reduce5< 16><<< dimGrid, dimBlock, smemSize >>>(d_idata, d_odata); break;
  case 8:
    reduce5<  8><<< dimGrid, dimBlock, smemSize >>>(d_idata, d_odata); break;
  case 4:
    reduce5<  4><<< dimGrid, dimBlock, smemSize >>>(d_idata, d_odata); break;
  case 2:
    reduce5<  2><<< dimGrid, dimBlock, smemSize >>>(d_idata, d_odata); break;
  case 1:
    reduce5<  1><<< dimGrid, dimBlock, smemSize >>>(d_idata, d_odata); break;
}
```

Reduction #7: Multiple Adds / Thread

Replace load and add of two elements:

```
unsigned int tid = threadIdx.x;  
unsigned int i = blockIdx.x*(blockDim.x*2) + threadIdx.x;  
sdata[tid] = g_idata[i] + g_idata[i+blockDim.x];  
__syncthreads();
```

With a while loop to add as many as necessary:

```
unsigned int tid = threadIdx.x;  
unsigned int i = blockIdx.x*(blockSize*2) + threadIdx.x;  
unsigned int gridSize = blockSize*2*gridDim.x;  
sdata[tid] = 0;  
  
while (i < n) {  
    sdata[tid] += g_idata[i] + g_idata[i+blockSize];  
    i += gridSize;  
}  
__syncthreads();
```

Can actually improve it further by using a local register variable instead of sdata[tid] and storing the result in sdata[tid] afterwards ("cascaded algorithm")!

Persistent Blocks

Number of blocks = #MP x #resident blocks = const for the GPU

Typical results of different optimization strategies (from a similar study)

KERNEL	TIME (S)	STEP SPEEDUP	CUMULATIVE SPEEDUP
Neighbored (divergence)	0.011722		
Neighbored (no divergence)	0.009321	1.26	1.26
Interleaved	0.006967	1.34	1.68
Unroll 8 blocks	0.001422	4.90	8.24
Unroll 8 blocks + last warp	0.001355	1.05	8.65
Unroll 8 blocks + loop + last warp	0.001280	1.06	9.16
Templatized kernel	0.001253	1.02	9.35

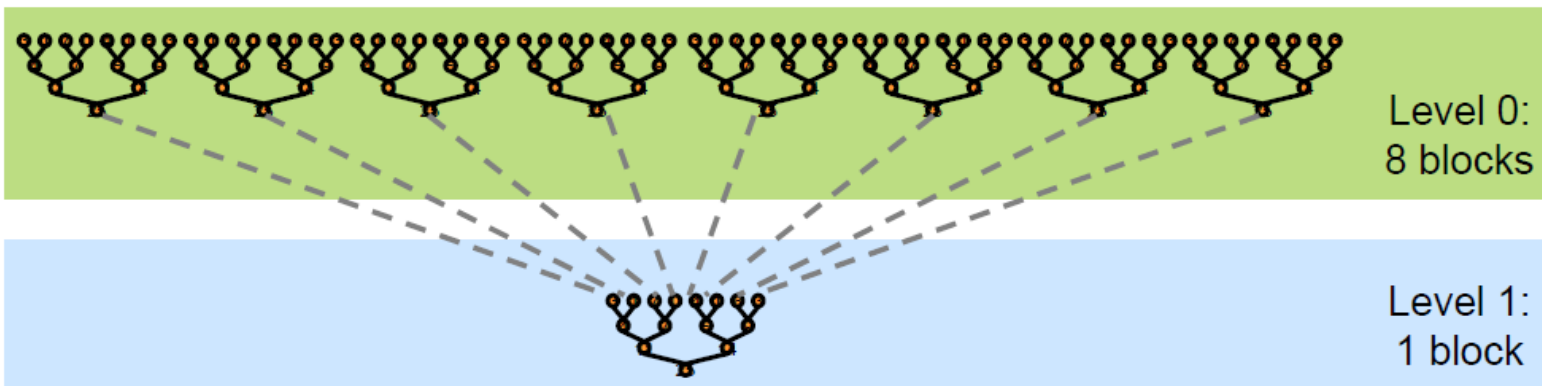
KERNELS	ELAPSED TIME (MS)	READ DATA ELEMENTS	WRITE DATA ELEMENTS	TOTAL BYTES	BANDWIDTH (GB/S)
reduceGmem	2.1357	16777216	131072	67633152	31.67
reduceSmem	1.1206	16777216	131072	67633152	60.35
reduceSmemUnroll	0.4171	16777216	32768	67239936	161.21
reduceSmemUnrollDyn	0.4169	16777216	32768	67239936	161.29

Time (%)	Time	Calls	Avg	Min	Max	Name
24.95%	4.0000us	1	4.0000us	4.0000us	4.0000us	reduceSmem()
17.76%	2.8480us	1	2.8480us	2.8480us	2.8480us	reduceShfl()

Final Stage of the Reduction

Treatment of larger data sets:

- multi-staged reduction kernel calls



- final reduction on CPU
- final reduction on GPU using atomic add function
- final reduction on GPU using threadfence function

Weakly-Ordered Memory Model of CUDA

```
__device__ volatile int X = 1, Y = 2;
__device__ void writeXY()
{
    X = 10;
    Y = 20;
}

__device__ void readXY()
{
    int A = X;
    int B = Y;
}
```

In a **weakly-ordered** memory model (CUDA) it is possible that $B = 20$ and $A = 1$

In a **strongly-ordered** memory the only options:

- $B = 20$ and $A = 10$
- $B = 2$ and $A = 1$
- $B = 2$ and $A = 10$

Memory Fence Functions

Memory fence functions enforce read/write ordering within different scopes (but they do not guarantee that all preceding memory functions actually occurred, in contrast to the synchronization barriers!)

There are memory fences on different hierarchy levels: block, grid, host-devices (`__threadfence_block()`, `__threadfence()`, `__threadfence_system()`)

`__threadfence()` waits until all global and shared memory accesses made by the calling thread prior to `__threadfence()` are visible to:

- all threads in the thread block for shared memory accesses,
- all threads in the device for global memory accesses

It is not necessary for each of the threads inside a threadblock to reach the memory fence (in contrast to the synchronization barrier)

```

__device__ unsigned int count = 0;
__shared__ bool isLastBlockDone;
__global__ void sum(const float* array, unsigned int N,
                   float* result)
{
    // Each block sums a subset of the input array
    float partialSum = calculatePartialSum(array, N);

    if (threadIdx.x == 0) {

        // Thread 0 of each block stores the partial sum
        // to global memory
        result[blockIdx.x] = partialSum;

        // Thread 0 makes sure its result is visible to
        // all other threads
        __threadfence();

        // Thread 0 of each block signals that it is done
        unsigned int value = atomicInc(&count, gridDim.x);

        // Thread 0 of each block determines if its block is
        // the last block to be done
        isLastBlockDone = (value == (gridDim.x - 1));
    }
}

```

```

// Synchronize to make sure that each thread reads
// the correct value of isLastBlockDone
__syncthreads();

```

```

if (isLastBlockDone) {

    // The last block sums the partial sums
    // stored in result[0 .. gridDim.x-1]
    float totalSum = calculateTotalSum(result);

    if (threadIdx.x == 0) {

        // Thread 0 of last block stores total sum
        // to global memory and resets count so that
        // next kernel call works properly
        result[0] = totalSum;
        count = 0;
    }
}
}

```

Homework Assignment 8 (3 points, due on 22.01.26)

For a floating point vector of 40MB size:

- Write a CPU implementation of the reduction algorithm (sum of all the elements). Use it for checking all of GPU outputs!!!
- Implement the reduction algorithm on GPU using the “cascaded algorithm” of Harris with the fixed number of blocks (approximately equal to the number of active blocks on your GPU) and all the optimizations discussed today. Include in kernel the final stage of reduction using the threadfence function.
- Compare its performance with the CPU result and with the fastest version of the atomic reduction kernel implemented in the previous homework. Analyze the results.
- Follow the usual homework submission guidelines.